Writing the Standards Down

Lab 2.a — the governance documents an AI agent actually uses

FSL AI Training · Phase 1 · Week 3

TODAY

Two hours, mostly hands-on

6 min	**Why**	What shared standards buy you
18 min	**The five artifacts**	What each is — and how to tell them apart
10 min	**How they get authored**	When one is required, who reviews, drafting with Claude
26 min	**Demo an ADR — then write yours**	The repository boundary, documented twice
44 min	**Derive + write an NFR**	Judge three fixes from last week — your verdict becomes the team standard
16 min	**Wire it in + review & retro**	Trigger table, binding line, smoke test, reflect

WHY

Sixteen developers, sixteen private setups

The assessment called this a Cultural Gap: engineers ahead of shared norms.

WHAT WORKS TODAY

- Individual practice is genuinely strong
- Claude is productive on this repo
- The habits from 1.b landed

WHAT DOES NOT

- None of it is shared
- Every dev re-teaches Claude the same things
- No way to tell if a rule is followed

WHAT TODAY CHANGES

- Habits become written rules
- Rules get homes: documents
- Written + agreed = a standard

And the evidence says authorship matters: an ETH Zurich study (Feb 2026) found LLM-generated context files LOWERED task success ~3% while raising cost 20%+; human-written improved it ~4%. /init-and-walk-away is worse than nothing.

SCOPE

Every rule has a reach — that reach is its layer

A rule is the agreement; a document is its home — one document can hold many rules. A rule's LAYER is its reach: who must obey it. That decides which document it belongs in, so the right people (and agents) inherit it.

Layer	Who must obey it	A real rule at this layer	So the document lives…
Collective	Everyone, in every repo	"Never commit secrets"	In an org-wide home — which does not exist yet
Project	Everyone in ONE repo	"Responses only via the respond.ts envelope"	In this repo: CLAUDE.md, docs/adr, docs/nfr
Feature / path	Only work touching certain files	"Server tests MUST use makeTestDb()"	In .claude/rules/testing.md — loads only on matching paths

Spot the mismatch: "never commit secrets" binds every repo, but it lives in THIS repo's CLAUDE.md — a Collective-reach rule parked at Project layer, because no org-wide home exists. Building that home is the sixth artifact 2.b hands your team.

THE ARTIFACTS

Five types — and the question each answers

Two acronyms spelled out once, then never again. If you can name the question, you know which document you are writing.

Artifact	The question it answers	What it is
CLAUDE.md	What must the agent know before anything else?	Always-loaded context + pointers
ADR	WHY is it like this?	Architecture Decision Record — one decision, with its cost
NFR	What must always be TRUE — and how do we check?	Non-Functional Requirement — a standing, testable rule
Best-practice recipe	HOW do I do X here?	A worked procedure — "recipe" from here on
Rules file	What applies ONLY in these files?	Path-scoped context (.claude/rules/)

The pair people conflate — NFR vs recipe: an NFR BINDS (what must be true, checkable; a deviation is a defect). A recipe GUIDES (the recommended route there; deviate freely, so long as the NFRs still hold).

DIFFERENTIATION

One subject, five artifacts

This repo has a real rule: only repositories/ may import db/store.ts. Here is what each artifact says about that same subject.

Artifact	What it says about the repository boundary
CLAUDE.md	The real line, verbatim: "Don't: access db.json outside server/src/repositories/". Always loaded, no rationale
ADR	Why a repository layer was chosen, what else was considered, what it costs. Written once
NFR	The rule as a standing requirement — plus how compliance is checked
Recipe	docs/best-practices/adding-a-resource.md — the steps for adding a resource within that boundary
Rules file	Guidance that loads only when you are in the files it applies to

None of them is redundant. ADR = history. NFR = law. Recipe = procedure. CLAUDE.md = index. Rules file = context that arrives just in time.

DIFFERENTIATION

Telling them apart when it is unclear

	CLAUDE.md	Rules file	ADR	NFR	Recipe
Answers	What to know first	What applies here	Why	What must hold	How
Tense	Present, standing	Present, scoped	Past — a choice made	Present — holds now	Imperative
Lifecycle	Revised continuously	Revised in place	Superseded, never edited	Revised in place	Updated freely
Carries a check?	No	No	No	**YES — that is why**	No
When it loads	**ALWAYS**	On matching paths	On demand, via a trigger	On demand + binding line	On demand, via a trigger
Lose it and…	The agent is unoriented	Guidance stops arriving	You lose the why	You lose the rule	You lose the how

Quick test: one-time choice → ADR · always-holds + checkable → NFR · steps → recipe · every session → CLAUDE.md · some files → rules file. One home per fact: duplicates drift — hence CLAUDE.md holds a trigger table, not the docs' contents.

How big should it get?

The scaling answer, because "keep it short" stops being useful advice at some point. The trigger table is Lab 1's lazy loading, systematized: pointer = the lazy load, trigger = when it fires.

THE HONEST ANSWER

- Lab 1.a said 100–200 lines. Right — to start.
- Past that, do not grow the file. Add a trigger row.
- The table is the mechanism: always-loaded index, on-demand detail.
- Dozens of docs later, copies drift: keep ONE home and let agents query it on demand.

CLAUDE.MD, THIS REPO

```
## Reference docs — read on demand

| When you are…      | Read first        |
| ------------------ | ----------------- |
| Changing response  | docs/adr/0001…    |
| shapes or codes    |                   |
| Calling an         | docs/nfr/0001…    |
| external API       |                   |

# a trigger is a SITUATION,
# not a topic
```

MATURITY

Start light — and know when to graduate

This repo ships the light forms. Today you write light ADRs — and an NFR that is already one rung up, because per-clause checks are the point.

	ADR	NFR
Start here	Nygard: Context · Decision · Status · Consequences	Prose rule + one "how compliance is checked" section
Graduate to	MADR: + Decision Drivers, Considered Options	Numbered MUSTs + per-clause checks + thresholds
Graduate when	Consequences keep coming out thin	The rule starts being ignored — you want a gate, not a hope

An NFR without a verification method is a wish with a document number.

AUTHORING

When is a document required — and who owns it?

The reason ADR practices die is not the template — nobody wrote down the trigger, the reviewer, or where it lives.

Write a…	When…	Authored / reviewed by
ADR	hard to reverse, contested, or a newcomer would ask "why is it like this?"	Anyone; peer PR review
NFR	the rule spans features AND you can say how it is checked	A few author, many adopt; EM reviews
Recipe	you have answered the same question twice	The person who answered; peer review
CLAUDE.md line	the agent needs it every session — and it is one line	Champion-gated: it costs every session
Rules file	it only applies when specific files are open	Same as a recipe

Peer review via PR is the acceptance gate today. An unreviewed standard is a draft. The "how we use these" README (2.b) exists to record exactly this table for your team.

AUTHORING

Drafting with Claude, honestly

The ETH finding again: generated context hurts, human-written helps. Draft, do not delegate.

THE DIVISION OF LABOUR

- Claude drafts from EVIDENCE in the code.
- You own the Decision and the Consequences.
- Never ask it what the team should do.
- Read the draft as someone who disagrees.

PROMPT STARTERS (2 OF 5 — CARD IN HANDOUT)

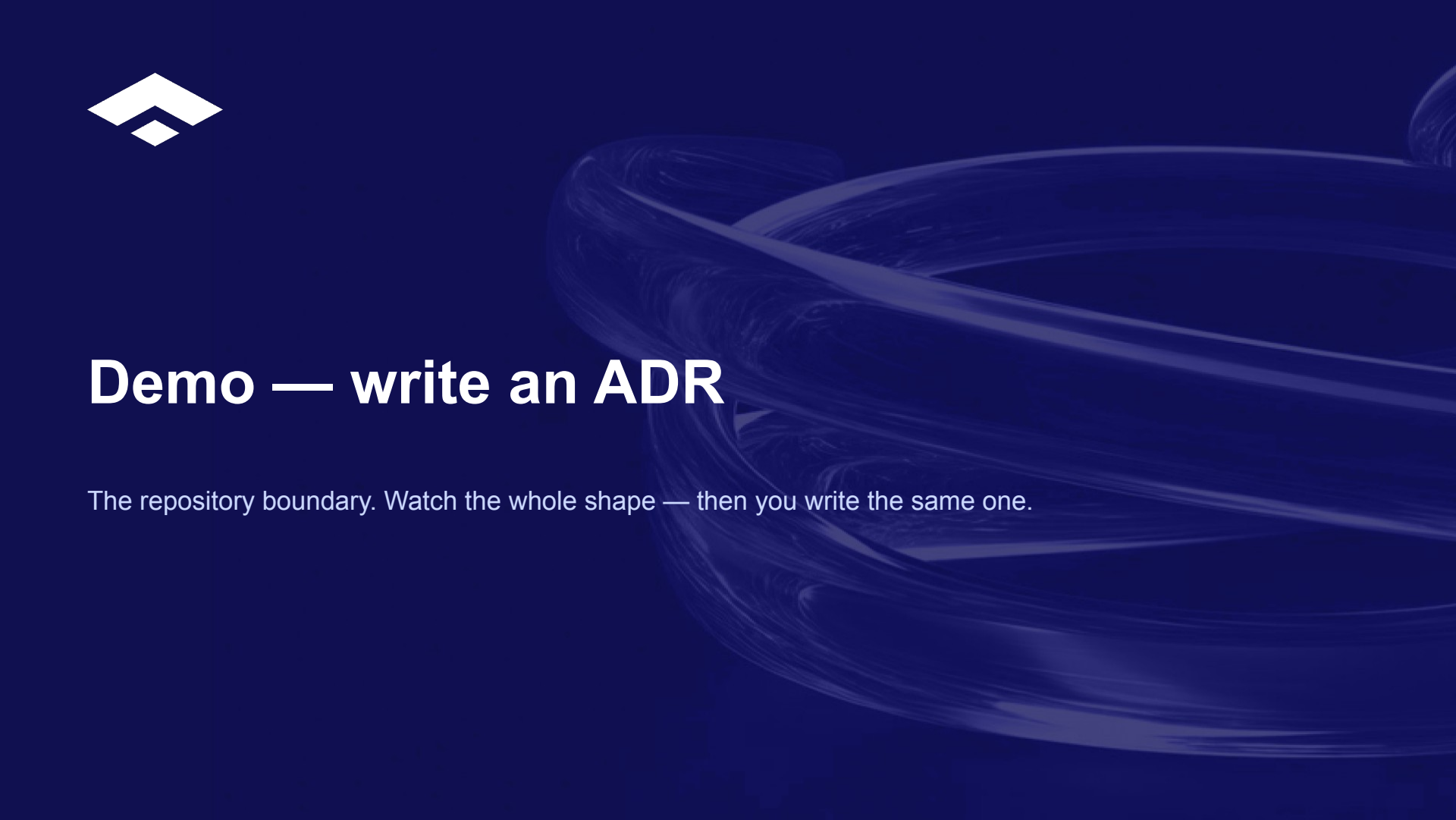
```
# gather evidence
Read @server/src/repositories/ and
@server/src/services/todos.service.ts.
What boundary do these files enforce,
and what would break if it were
violated? Don't propose changes.

# draft, but leave the decision
Using @docs/adr/template.md, draft
an ADR for that boundary. Fill
Context and Consequences from what
you just read. Leave Decision as a
question for me to answer.
```

LIFECYCLE

Keeping them true

The half of governance nobody plans for: what happens when the world moves.

- **Supersede, never edit, an accepted ADR.** The old decision happened. Write a new one that replaces it.
- **Revise NFRs in place.** They describe what holds now.
- **Every rule cites the incident that produced it.** A "Why" naming a real failure survives staff turnover; "best practice" does not.
- **Review on a cadence, not on vibes.** A date on the document, and a person against it.



Demo — write an ADR

The repository boundary. Watch the whole shape — then you write the same one.

DEMO · THE GAP

A rule with no recorded reason

Real, load-bearing, and undocumented — the ADR trigger on all three counts.

- **The rule exists.** CLAUDE.md says routes and services never import `db/store.ts`.
- **The code obeys it.** Only `server/src/repositories/` touches the store.
- **Nothing says why.** A newcomer sees a constraint with no rationale — so they route around it.
- **Nothing checks it.** Which is next week's problem, and worth noticing now.

> "But the decision was already made — isn't an ADR written when you decide?" Recording is not deciding: a retroactive ADR captures the WHY before it evaporates with the people who knew it. Status: Accepted. Date: when recorded, not when decided.

DEMO · THE STEPS

Six steps — follow along on screen

Steps 2 and 4 are Claude's; steps 3, 5, and 6 are yours — that split is the lesson. The full walkthrough, with prompts and example replies: docs/labs/lab-2a-demo-walkthrough.md.

#	What I do	What you should see
1	Create the file from the template	docs/adr/0002-repository-only-store-access.md — next number, kebab-case title
2	Prompt 1 — gather evidence	Claude reads repositories/ + a service, reports the boundary. It READS; it does not decide
3	Check its answer against the code	If Claude got the boundary wrong, correct it now — bad evidence makes a bad Context
4	Prompt 2 — draft, Decision left open	Context + Consequences filled from evidence; Decision returned as a question to me
5	Edit Context; WRITE the Decision myself	One sentence, present tense: "Only modules under repositories/ may import db/store.ts"
6	Add one honest cost, then stage it	A consequence someone could complain about — then git add

DEMO · THE RESULT

What good looks like

The finished artifact. The interesting part is the Consequences section.

THE SPLIT THAT MATTERS

- Claude gathered the evidence. I edited it.
- The Decision is ONE sentence, and it is mine.
- Context has no verbs of choosing — it describes forces.
- At least one consequence is genuinely a cost.

MATCHES DOCS/ADR/TEMPLATE.MD

```
# docs/adr/0002-repository-only-
#   store-access.md

- Status: Accepted
- Date: <today>
- Deciders: <the team>

## Context
<from Claude, edited>

## Decision
Only modules under
server/src/repositories/ may import
db/store.ts.  <-- I wrote this

## Consequences
- Good: persistence is swappable
- Bad: one more hop for simple reads
```

YOUR TURN

Now write yours — same subject, your words

Everything today is yours, in your copy — including the clause list you derive later.

- **Ten minutes, in your own copy.** Same boundary, your own Context and Consequences. The motion is the point, not novelty.
- **Use the two prompts you just watched.** Evidence first, then draft-with-the-decision-left-open. Both are in the handout.
- **Write the Decision yourself.** One sentence. If it came out of Claude, rewrite it.
- **Find one honest cost.** The acceptance bar has a cost in Consequences — "none" is not an answer.

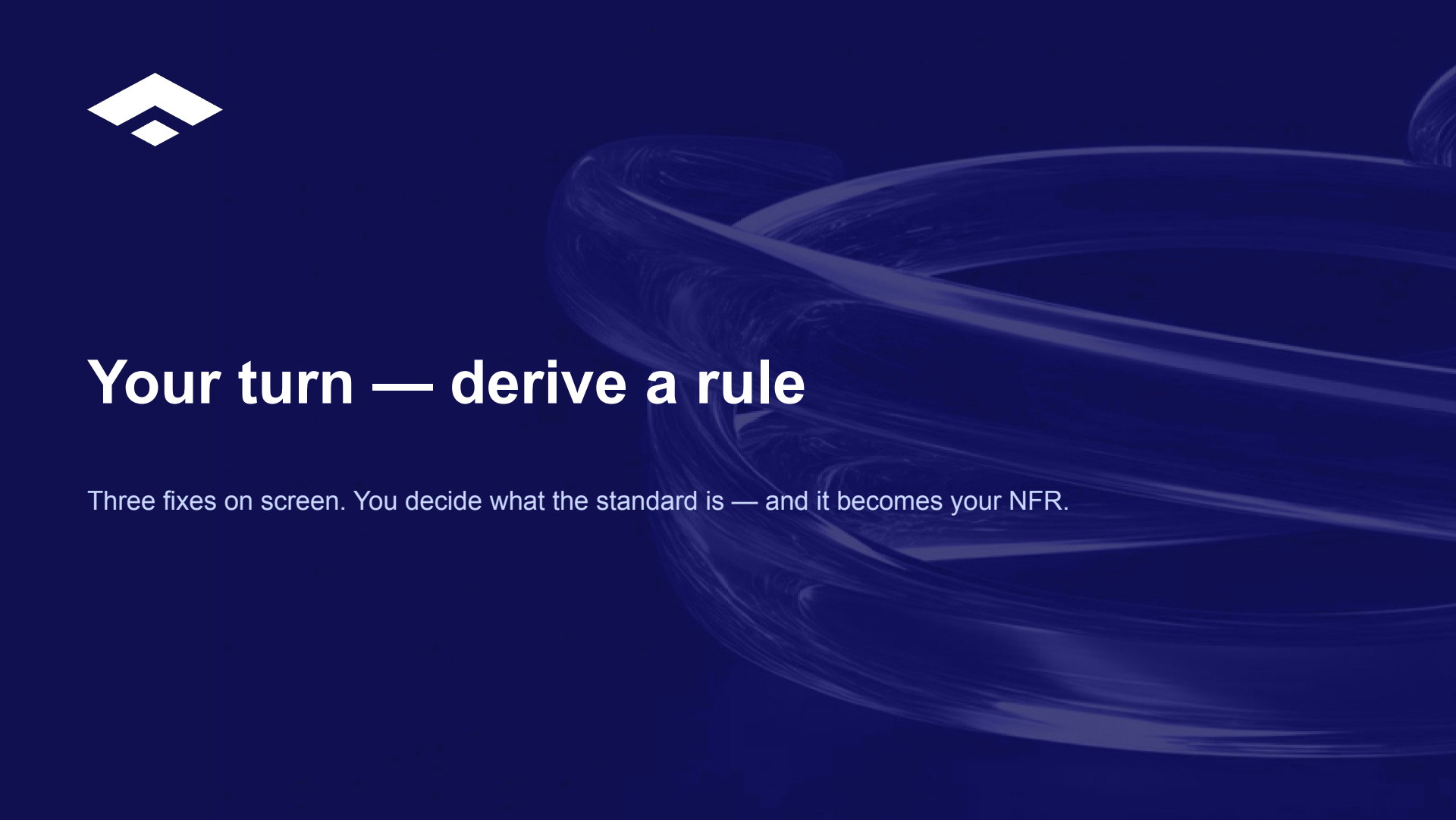
CHECKPOINT

Checkpoint 1

NOBODY ADVANCES UNTIL THIS IS TRUE

You have docs/adr/0002-*.md staged in YOUR copy: metadata block (Status / Date / Deciders) complete, all three sections — Context, Decision, Consequences — filled, and at least one real cost.

Paste your Decision sentence into chat. If it came out of Claude rather than out of you, rewrite it first — that is the one part of an ADR that cannot be delegated.



Your turn — derive a rule

Three fixes on screen. You decide what the standard is — and it becomes your NFR.

STIMULUS

Three ways the same bug got fixed

Three fixes, authored for this exercise. All reviewed. All merged. All green. This is the raw material for the rule you are about to write.

Fix	The situation	CI
A	Fixed under pressure and shipped. "Test to follow" — it never did.	green
B	Fixed, then a test written afterwards. It passes — and it also passes on the UNfixed code.	green
C	A failing test written first; the fix made it pass. (This is what your own 1.b PRs did.)	green

While you read, answer two questions per fix, silently: Would you merge it? And — what, exactly, convinces you the bug is actually fixed? Hold your answers.

DERIVE · THE CLOCK

Your ten minutes, structured

A good clause names an observable fact: "a change under server/src ships with a test in the same PR" is a clause. "Tests should be good" is a wish.

Min	Do this	Output
0–4	Verdict each seed: keep, tighten, or drop. A clause earns KEEP only if you can name which of A / B / C it catches	Four verdicts, each tagged MECHANICAL or FUZZY
4–7	Add your own clauses — what did the seeds miss?	One or two more, tagged. More than 6 total means they're too small
7–9	Mark the one clause you're LEAST sure about	That is what you'll raise at the compare — doubt is the interesting output
9–10	Paste your list into the shared Doc, under your name	Everyone's lists, side by side — the raw material for the class standard

All of it is yours: this list, and the NFR you write from it — your own copy, your own PR. The class compare that follows is calibration, not a committee. (And: tighten = more checkable, not more strict.)

DERIVE

Which are acceptable? Write the rule.

Ten minutes, on your own, on the clock you just saw: verdict each seed (keep / tighten / drop), tag it, add your own. These clauses become the Rules section of your NFR.

YOUR SEED CLAUSES

- 1. A fix ships with a test
- 2. The test failed before the fix
- 3. Test lives next to its module
- 4. Full suite green before the PR

TAG EVERY CLAUSE

- MECHANICAL — a script decides it from the diff
- FUZZY — it needs a person
- The tag is the point. Do not skip it

THEN THE COMPARE

- Paste your list in the Doc, under your name
- The class agrees ONE list from them
- Stretch: the store boundary, same method

CONVERGE

Where we landed

Your lists, side by side in the Doc — now the class agrees ONE. Written down before anyone moves on.

MECHANICAL

- A change under server/src ships with a test
- The test file is co-located
- Full suite green

FUZZY

- The test failed first
- The test asserts behaviour, not implementation
- The assertion is meaningful

DECIDE NOW

- Accepted risk?
- Human gate?
- Or dropped?

Which of the five artifacts is this list? Apply the quick test: must always hold, and you can say how it is checked → an NFR. It feels decision-ish — but it is present-tense law, revised in place. Tiebreak when lists conflict: the stricter clause wins unless someone shows it false-positives on real code here.

THE HINGE

Same file, three enforcement states

The lesson: WRITTEN and ENFORCED are different properties of a rule. Three rules from the CLAUDE.md you already trust — one in each state.

ONE CLAUDE.MD, THREE RULES — ONE RULE IN EACH STATE

- WRITTEN ONLY — the store boundary: nothing checks it. A promise.
- CHECKED — console.log: lint fails it. A gate.
- HALF-CHECKED — logger.info with a message where the file basename belongs: types pass, lint passes. The dangerous one — it LOOKS enforced.
- Every clause you just tagged sits in one of these states. The Checked-by line in your NFR records which.

THE EVIDENCE — EVERY CHECKER'S VERDICT, PER CALL

```
// the rule (CLAUDE.md):
// logger.info('<file-basename>',
//   message, meta?)

console.log('created', id)
// LINT ERROR (no-console).
// The channel is CHECKED.

logger.info('creating a todo',
  'created', { id })
// tsc: clean. eslint: clean.
// The scope convention is
// checked by NOTHING.
```

COMPARE

What a mature team wrote

A real NFR from a mature 137-document corpus — shown now that your list exists, not before.

Section	What it contains
Classification	Category, priority, and which external framework it traces to
Requirements	Numbered MUST clauses — no prose, no hedging
Thresholds	Target / Warning / Critical, so "compliant" is measurable
Verification	Per clause: the exact test or check that proves it
Plan reference	Which phase implements it, so it is not aspirational

What did you catch that they did not? Their form is heavier than you need today; the per-clause Verification is the part worth stealing.

WRITE IT UP

Turn the clauses into an NFR

Twenty minutes: your own doc, in your own copy, from the agreed clause list.

THE RULES

- Copy NFR-0001's headings and tone —
- then ADD the per-clause checks it predates.
- Every clause: "Checked by: <what>".
- Unenforceable ones get the marking convention →
- Peer review is the last five minutes. Protected.

DOCS/NFR/0002-… (YOURS)

```
## The rules

1. A change under server/src ships
   with a test in the same PR.
   Checked by: CI diff check.

2. The test failed before the fix.
   Checked by: — NOT MECHANICALLY
   CHECKABLE — decision: human gate
   (reviewer asks) — <name>

# clause + check + mark + name.
# an unmarked unenforceable clause
# is the dangerous kind.
```

CHECKPOINT

Checkpoint 2

NOBODY ADVANCES UNTIL THIS IS TRUE

Your NFR names how each clause is checked — and the ones that cannot be are marked, with a decision and a name.

Artifact check, not a state check: I am reading your document, not your test output. A clause with no check and no mark is the one thing that fails here.

WIRE IT IN

A doc nothing points at is a doc nobody reads

Claude included. This step is why the last two hours matter.

THE SAME RULE, WIRED TWICE — TWO DIFFERENT JOBS

- BINDING LINE — enforces by PRESENCE: it IS the rule's headline sentence, riding in with CLAUDE.md every session. No fetch DECISION.
- TRIGGER ROW — a SITUATION ("when you are…") plus a POINTER ("read first"): the situation says when, the pointer loads the FULL doc.
- When does a rule EARN a binding line? Name its situation. Specific ("calling an external API") — trigger row only. "Every change" — binding line.
- The (NFR-0002) tag is a citation, not a loader.

CLAUDE.MD

```
| When you are…       | Read first      |
| ------------------- | --------------- |
| Changing the store  | docs/adr/0002…  |
| boundary            |                 |
| Fixing a bug or     | docs/nfr/0002…  |
| adding behaviour    |                 |

## Do / Don't
- Do: every change under server/src
  ships with a test in the same PR
  (NFR-0002)   <-- the binding line

# smoke test, fresh session:
# "what is our rule about tests?"
```

WIRE IT IN

One rule, one home, four references

The NFR document is the one home of the full law. Everything else points at it — each reference with a different job.

Reference	The line itself	What it does
Binding line · layer 1	`- Do: every change under server/src ships with a test in the same PR (NFR-0002)`	States the rule, always present. Informs every session
Trigger row · layer 1	`| Fixing a bug or adding behaviour | docs/nfr/0002... |`	Loads the full NFR when its situation matches
Hook · layer 2 — next week	`(not in CLAUDE.md: .claude/hooks/ + settings.json)`	Nudges the agent the moment an edit violates the clause
CI gate · layer 4 — next week	`(not in CLAUDE.md: .github/workflows/governance.yml)`	Fails the PR that violates the clause — stops anyone

Layer 1 informs; layers 2–4 enforce — next week builds the bottom two rows for this same clause. Nothing built today validates anything.

REVIEW & RETRO

Compare, reflect, capture

Five minutes that make the difference between an exercise and a practice.

- **Two volunteers, two NFRs.** Same clause list, different documents — screen-share each: what did they choose to check, and what did they mark?
- **One retro question.** What was hard, what was easy? One line each in chat, waterfall style.
- **Reflection note — now, not later.** 3–5 sentences in your PR description: what you learned, what surprised you, what you'd apply tomorrow. It is an acceptance criterion.
- **Unfinished work finishes async.** Still required, still peer-reviewed — the list does not shrink because the clock ran out.

WRAP

Ship checklist, and what is next

Questions & support: #extra-duty-solutions-ai-training — your instructor and the Applied AI Engineer are there.

- **Ship checklist.** ADR (cost included) · NFR (checks + marks) · wired into CLAUDE.md (trigger rows + binding line) · smoke-tested · PR-reviewed, CI green · reflection note in the PR.
- **Stretch, self-paced.** Store-boundary clauses · the half-gate ESLint clause · a rules-file conversion · a head start on the "how we use these" README.
- **Everything is in the handout.** All five prompt starters, the stuck-points table, the skeletons.
- **Next week: teeth.** Your clause becomes a hook and a CI gate — and the marked clauses get their reckoning.